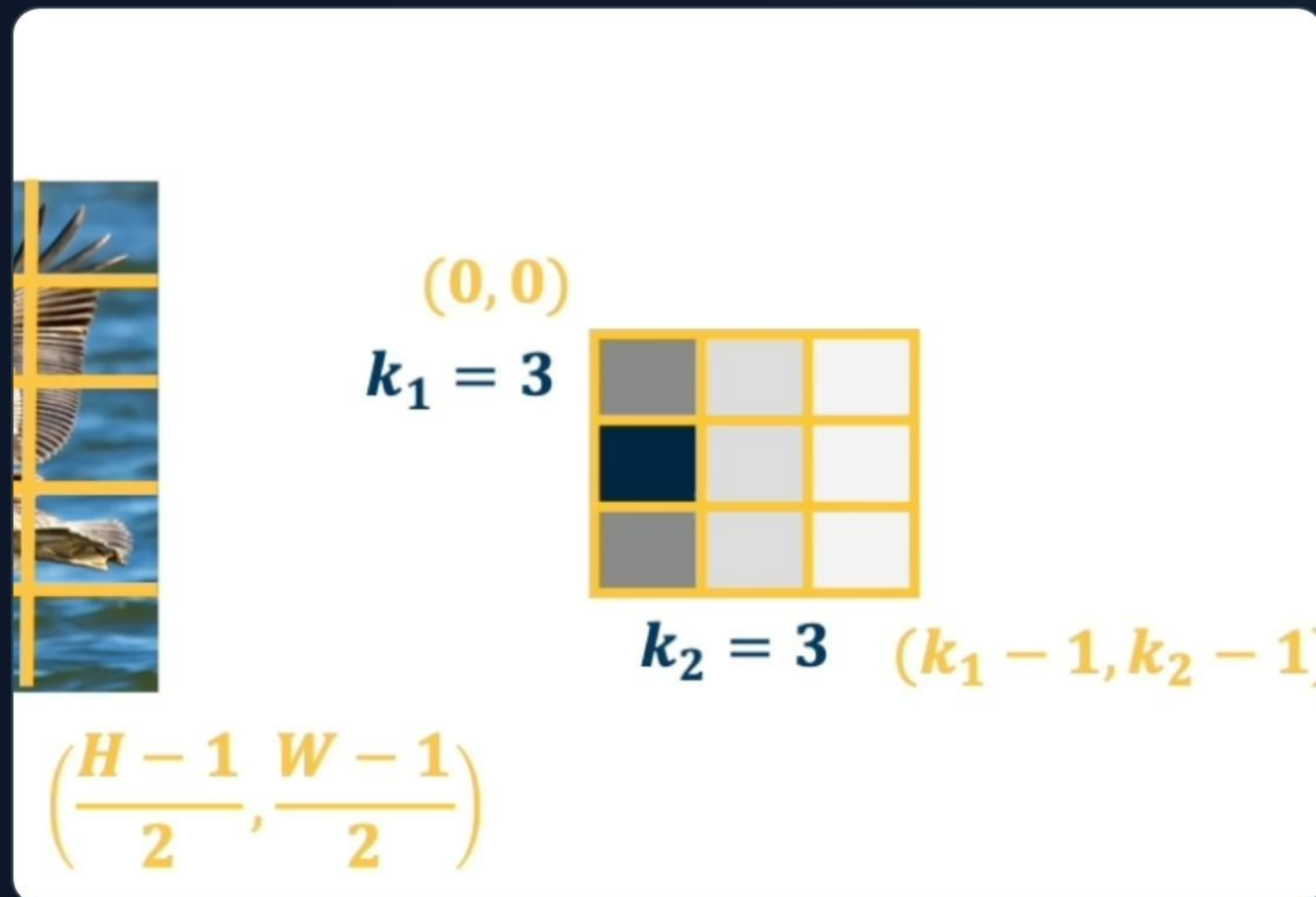


Beyond Summarization

Advanced Strategies for Overcoming LLM Context Window
Limitations in Modern Agentic Architectures

The Summarization Trap

- Summarization is the most obvious solution when a context window overflows, but it has severe systemic drawbacks.
- The primary issue is **information loss**. Crucial nuances, implicit references, and granular details are permanently erased.
- Modern LLMs and advanced agent frameworks use sophisticated methods to manage 50,000+ token histories without dropping data.
- If asked in an engineering interview, jumping straight to summarization can highlight a lack of depth in system architecture.



Context Filtering & Pruning



Selective Context

Don't send irrelevant info. If a user asks a Python error question, do not pass their entire chat history regarding SQL and Docker. Use a relevance scorer to fetch only the top related messages dynamically.



Context Pruning

Before sending the payload to the LLM, assign a direct importance score to every message in the active array. Keep only the highest-value items (e.g., messages 5, 28, and 99) and completely discard the noise.

Hierarchical Memory

Multi-Level Storage

Advanced agent systems store information at distinctly different levels rather than flattening everything into a single, massive string.

- **Level 1:** Recent messages (Immediate working space)
- **Level 2:** Important facts (Extracted key points)
- **Level 3:** Long-term archive (Historical logs)

Dynamic Retrieval

When the LLM processes a current query, the system architecture dynamically fetches and weaves together only the relevant sub-chunks from each of these hierarchical levels. This provides rich context without ever blowing up the token count.

Semantic Compression

Instead of summarizing conversational facts in English, modern systems convert key user attributes directly into embeddings.

By storing representations like 'User likes Python' as multi-dimensional vectors, you can accurately retrieve the precise context later. This robust semantic compression completely prevents data loss and behaves identically to vector databases.



State Tracking Systems

The 200-Message Problem

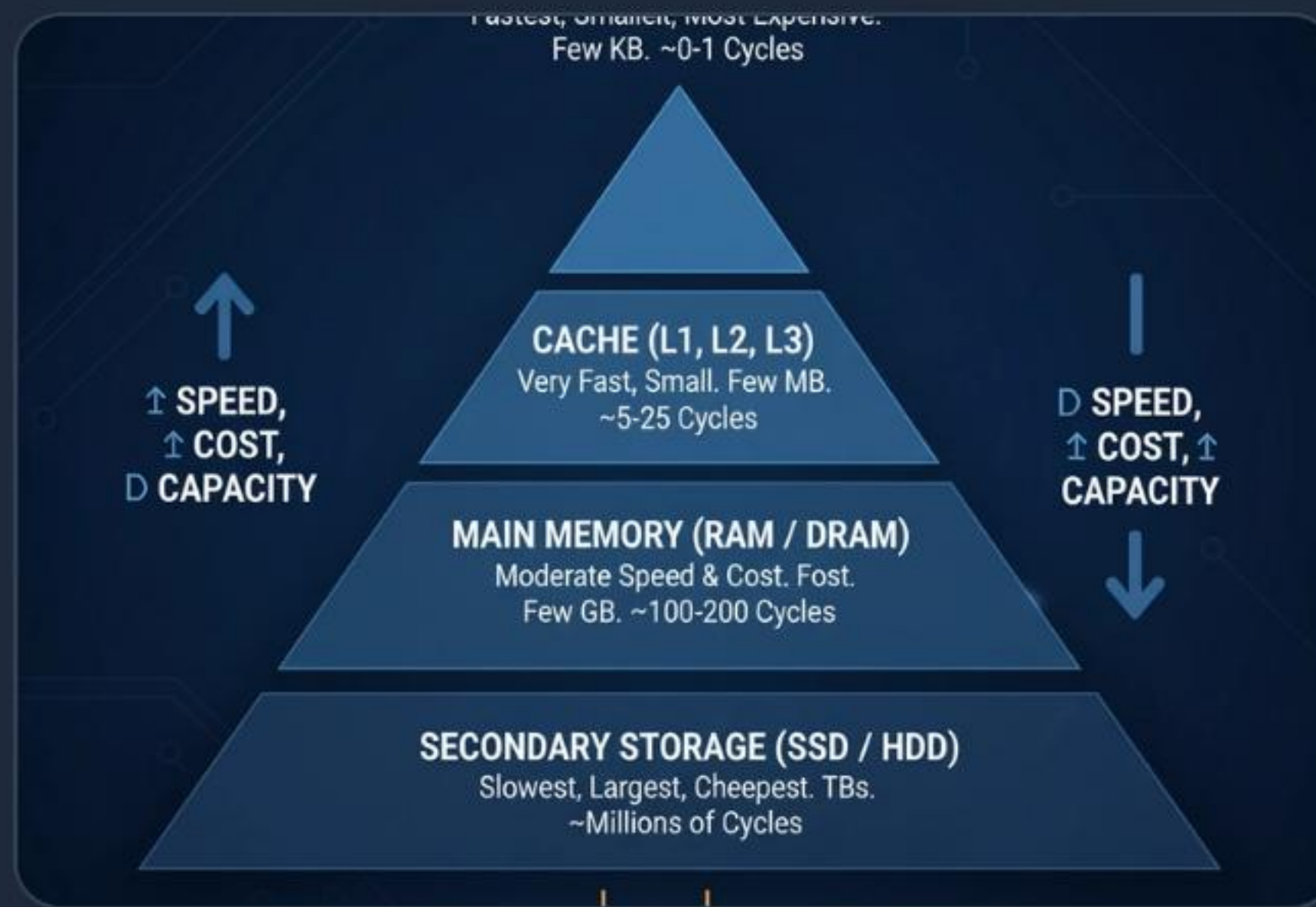
For transactional workflows, like a flight booking agent, retaining the sprawling conversation history is highly inefficient. Sending hundreds of conversational messages back and forth rapidly exhausts context windows.

The JSON State Solution

Maintain an active, condensed state object representing the workflow. The LLM only receives this token-efficient representation:

```
{  
  "destination": "Lahore",  
  "date": "June 30",  
  "passengers": 2  
}
```

Scratchpad & Memory Separation



Working Memory

Contains the current task context and the active, step-by-step reasoning scratchpad.



Selective Injection

The central orchestrator acts as a bridge, retrieving and injecting only the necessary nodes.

Long Term Vault

Long-Term Memory

Securely houses broader project details, global settings, and user preferences.

Query Rewriting

- A standard paradigm in advanced RAG pipelines relies on avoiding dumping massive context buffers at all costs.
- Instead of processing an ambiguous user prompt directly, the system intercepts and heavily rewrites it.
- **Original Prompt:** "What did we decide about the database issue?"
- **Rewritten Query:** `Search: database issue decision`
- This drastically narrows down the exact information retrieved rather than demanding the LLM read through unstructured raw logs.



Multi-Agent Decomposition



Avoid the Mega-Prompt

Putting instructions for Research, Coding, Testing, and Documentation together into one massive context window inevitably leads to critical context explosion and severe cognitive overload for a single LLM process.



Split the Workforce

Divide the overarching workload: Agent 1 researches, Agent 2 designs, Agent 3 codes. Because each specific agent executes completely independently, they only require a tiny fraction of the overall context to succeed.

Attention-Based Context Ranking

Leading research systems refuse to blindly send all retrieved text chunks. Instead, they strictly rank chunks via attention-guided algorithms, injecting only the absolute highest-value signals into the prompt.

DATA CHUNK IDENTIFIER	EXTRACTED CONTENT TYPE	RELEVANCE SCORE	SYSTEM ACTION
Chunk A	Direct Error Log Match	0.95	✓ Injected into Prompt
Chunk B	Recent Stack Trace	0.88	✓ Injected into Prompt
Chunk C	Historical Code Refactor	0.12	✗ Discarded Safely
Chunk D	Unrelated Docker Ticket	0.04	✗ Discarded Safely

Tool Calling Over Context

The Missing Technique

Many developers miss this highly critical optimization: immediately stop passing the entire raw database schema or massive arrays of user records directly into the LLM's context simply to await a user query.

Precision Execution

Instead, equip the LLM explicitly with tools like `get_customer_order()`. The LLM only triggers the function and fetches precisely what is requested (e.g., `{ order_id: 123, status: "Delivered" }`). This dramatically slashes token burn.

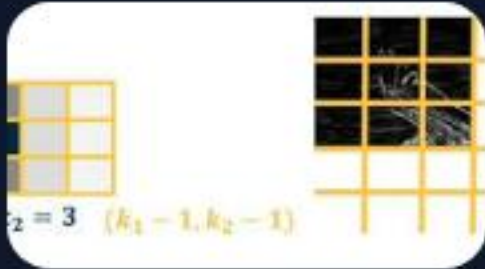


I would first try to avoid sending unnecessary context through retrieval, relevance filtering, state management, tool calls, hierarchical memory, and context pruning. Summarization is useful, but it should generally be the last resort because it introduces information loss.



THE IDEAL ENGINEERING RESPONSE

Image Sources



https://lowyx.com/assets/posts/gatech/dl/m2l5_convolution_coordinates.png

Source: lowyx.com



https://static.vecteezy.com/system/resources/previews/060/615/203/non_2x/blue-futuristic-technology-abstract-background-with-glowing-hexagonal-nodes-data-visualizations-modern-digital-network-concept-vector.jpg

Source: www.vecteezy.com



<https://media2.dev.to/dynamic/image/width=800%2Cheight=%2Cfit=scale-down%2Cgravity=auto%2Cformat=auto/https%3A%2F%2Fdev-to-uploads.s3.amazonaws.com%2Fuploads%2Farticles%2Fu20cpss6g2uqdh43cefr.png>

Source: dev.to



https://static.vecteezy.com/system/resources/previews/069/165/001/non_2x/digital-circuit-board-with-glowing-dotted-lines-futuristic-dark-background-with-blue-and-red-data-routes-for-cybersecurity-tech-ui-machine-learning-visuals-illustration-vector.jpg

Source: www.vecteezy.com



<https://cloudzy.com/wp-content/uploads/2026/02/1-scaled.webp>

Source: cloudzy.com



https://media.istockphoto.com/id/2000515525/photo/magnifying-glass-is-searching-for-the-red-word-spyware-among-green-binary-numbers.jpg?s=612x612&w=0&k=20&c=IQro4gobSvuz_B3dvT7PB4u9WGrrwOgdUfptN-fasP4=

Source: www.istockphoto.com